

Clase viernes 22 de mayo

Hackaton — Implementación simplificada de un asignador dinámico de memoria

En esta actividad trabajaremos sobre una versión simplificada de un asignador dinámico de memoria similar a `malloc()` y `free()`.

El objetivo NO es implementar un `malloc` real de Linux o glibc, sino comprender algunos conceptos fundamentales de administración de memoria contigua:

- representación de bloques libres y ocupados;
 - fragmentación externa;
 - algoritmos de búsqueda de huecos;
 - liberación de memoria;
 - fusión de huecos adyacentes.
-

Primera etapa — Comprender la interfaz y el código suministrado

Durante la clase se mostrará una primera versión del programa usando el `malloc()` y `free()` reales de la biblioteca estándar.

Esa primera versión `mallocv1` utiliza una pequeña interfaz interactiva con los comandos:

```
m SIZE
f ADDRESS
q
```

El objetivo de esa primera versión es mostrar la interfaz esperada y cómo se utilizará el asignador.

Todos los fuentes se encuentran en el servidor del curso, en el directorio

`/var/local/ssoo2026/my_malloc`

Segunda etapa — Implementar nuestro propio asignador (a partir de código ya hecho)

El proyecto entregado reemplaza el uso de `malloc()` por:

`my_malloc()`

La memoria administrada por el asignador se obtiene mediante:

`mmap(...)`

y luego se divide internamente en bloques administrados por ustedes.

`mmap()` es una llamada al sistema que permite solicitar al sistema operativo una región contigua de memoria virtual. En este proyecto, se utilizará para reservar una gran “arena” de memoria sobre la cual implementaremos nuestro propio asignador dinámico (`my_malloc` y `my_free`). A diferencia de `malloc()`, que pertenece a la biblioteca estándar de C, `mmap()` interactúa directamente con el kernel del sistema operativo.

El proyecto ya incluye:

- estructuras y constantes;
- inicialización del asignador;
- una implementación básica de `my_malloc()`;
- algoritmo First Fit;
- división de bloques (`split_block`);
- un programa interactivo (`allocator_test`).

Actualmente, `allocator_test` solamente implementa el comando:

`m SIZE`

que realiza asignaciones de memoria usando `my_malloc()`.

A partir de este punto deberán extender el proyecto agregando nuevas funcionalidades.

Los archivos que comprenden el proyecto son:

- `Makefile`
- `my_malloc.h`
- `allocator_test.c`
- `my_malloc_init.c`

- `my_malloc.c`

Estructura interna del esquema de bloques ocupados / libres

Cada bloque contiene:

- un encabezado (`BlockHeader`);
- un área de datos utilizable por el usuario;
- un puntero al siguiente bloque;
- un indicador que informa si el bloque está libre u ocupado.

Todos los bloques forman una lista enlazada dentro de la arena de memoria solicitada al sistema operativo mediante `mmap()`.

Conceptualmente:

```
+-----+-----+
| BlockHeader      | datos del usuario  |
+-----+-----+
```

Los bloques forman una lista enlazada dentro de la propia arena de memoria.

Algo así:

Inicialmente:

```
arena_start
  ↓
+-----+-----+
| BlockHeader      | usable free memory  |
| size = arena_size |                    |
| is_free = 1      |                    |
| next = NULL      |                    |
+-----+-----+
```

Luego de algunas asignaciones/liberaciones:

Lista de usuarios			Lista de usuarios		
BlockHeader	datos del usuario	--->	BlockHeader	datos del usuario	---
size = 100			size = 200		
is_free = 0	OCUPADO		is_free = 1	LIBRE	
next -----+			next -----+		
+-----+			+-----+		
+-----+			+-----+		
BlockHeader	datos del usuario	--->	BlockHeader	datos del usuario	---
NULL					
size = 50			size = 300		
is_free = 0	OCUPADO		is_free = 1	LIBRE	
next -----+			next = NULL		
+-----+			+-----+		

Parte 1 — Visualización de memoria

Implementar una función que imprima el estado actual de la arena.

La visualización debe incluir:

- dirección lógica del bloque;
- tamaño;
- estado libre u ocupado.

Por ejemplo:

```
0x7f4a12000000 USED size=100
0x7f4a12000084 FREE size=200
0x7f4a12000170 USED size=50
```

0x7f4a120001c0 FREE size=300

o una representación equivalente.

El objetivo es poder observar:

- cómo se divide la memoria;
- qué bloques están libres;
- qué bloques están ocupados;
- cómo evoluciona la fragmentación;
- cómo cambian las direcciones de memoria.

Se recomienda invocar esta función después de cada operación.

Parte 2 — Implementación de `my_free()`

Implementar:

```
void my_free(void *ptr);
```

Inicialmente, la función solamente debe:

- localizar el bloque correspondiente;
- marcarlo como libre.

En esta primera etapa NO se debe fusionar bloques adyacentes.

Luego de liberar memoria, el mapa puede quedar así:

```
0x1000 USED size=100  
0x1084 FREE size=200  
0x1170 FREE size=300  
0x12b0 USED size=50
```

Parte 3 — Fusión de bloques libres

Implementar la fusión de huecos contiguos.

Si dos bloques libres son adyacentes en memoria, deben convertirse en un único bloque mayor.

Casos posibles:

1. No hay vecinos libres

[USED][FREE][USED]

No se realiza ninguna acción adicional.

2. Vecino libre a derecha

[USED][FREE][FREE]

Se fusionan ambos bloques libres.

3. Vecino libre a izquierda

[FREE][FREE][USED]

Se fusionan ambos bloques libres.

4. Vecinos libres a ambos lados

[FREE][FREE][FREE]

Los tres bloques deben fusionarse en uno solo.

Parte 4 — Nuevos algoritmos de asignación

Actualmente el proyecto usa:

First Fit

que selecciona el primer hueco suficientemente grande.

Implementar además:

Best Fit

Selecciona el hueco libre más pequeño posible que permita satisfacer el pedido.

Worst Fit

Selecciona el hueco libre más grande disponible.

Objetivo final

Al finalizar la actividad deberían poder:

- observar fragmentación externa;
- comparar distintos algoritmos de asignación;
- analizar el efecto de liberar memoria;
- visualizar la necesidad de fusionar huecos;
- comprender cómo un asignador administra bloques dentro de una gran región obtenida del sistema operativo.

No es necesario completar todas las partes durante la hackaton.

La solución completa será discutida posteriormente en clase.